
Architectural Design

Topics Covered

- Architectural design decisions
- System organisation
- Modular decomposition styles
- Control styles

Software architecture

- The design process for identifying the sub-systems making up a system and the framework for sub-system control and communication is **architectural design**.
- The output of this design process is a description of the **software architecture**.

Architectural design

- An early stage of the system design process.
- Represents the link between specification and design processes.
- Often carried out in parallel with some specification activities.
- It involves identifying major system components and their communications.

Advantages of explicit architecture

- Stakeholder communication
 - Architecture may be used as a focus of discussion by system stakeholders.
- System analysis
 - Means that analysis of whether the system can meet its non-functional requirements is possible.
- Large-scale reuse
 - The architecture may be reusable across a range of systems.

Architecture and system characteristics

- Performance
 - Localise critical operations and minimise communications. Use large rather than fine-grain components.
- Security
 - Use a layered architecture with critical assets in the inner layers.
- Safety
 - Localise safety-critical features in a small number of sub-systems.
- Availability
 - Include redundant components and mechanisms for fault tolerance.
- Maintainability
 - Use fine-grain, replaceable components.

Architectural models

- Different architectural models may be produced during the design process
- Each model presents different perspectives on the architecture

Architectural conflicts

- Using large-grain components improves performance but reduces maintainability.
- Introducing redundant data improves availability but makes security more difficult.
- Localising safety-related features usually means more communication so degraded performance.

Box and line diagrams

- Very abstract - they do not show the nature of component relationships nor the externally visible properties of the sub-systems.
- However, useful for communication with stakeholders and for project planning.

Architectural design decisions

- . Architectural design is a creative process where you try to establish a system organization that will satisfy the **functional and non-functional system requirements**
- It is a creative process, the activities within the process differ radically depending on the type of system being developed, the background and experience of the system architect, and the specific requirements for the system.

Architectural design decisions

- Is there a generic application architecture that can be used?
- How will the system be distributed?
- What architectural styles are appropriate?
- What approach will be used to structure the system?
- How will the system be decomposed into modules?
- What control strategy should be used?
- How will the architectural design be evaluated?
- How should the architecture be documented?

Architectural models

- Used to document an architectural design.
- Static structural model that shows the major system components.
- Dynamic process model that shows the process structure of the system.
- Interface model that defines sub-system interfaces.
- Relationships model such as a data-flow model that shows sub-system relationships.
- Distribution model that shows how sub-systems are distributed across computers.

System organisation

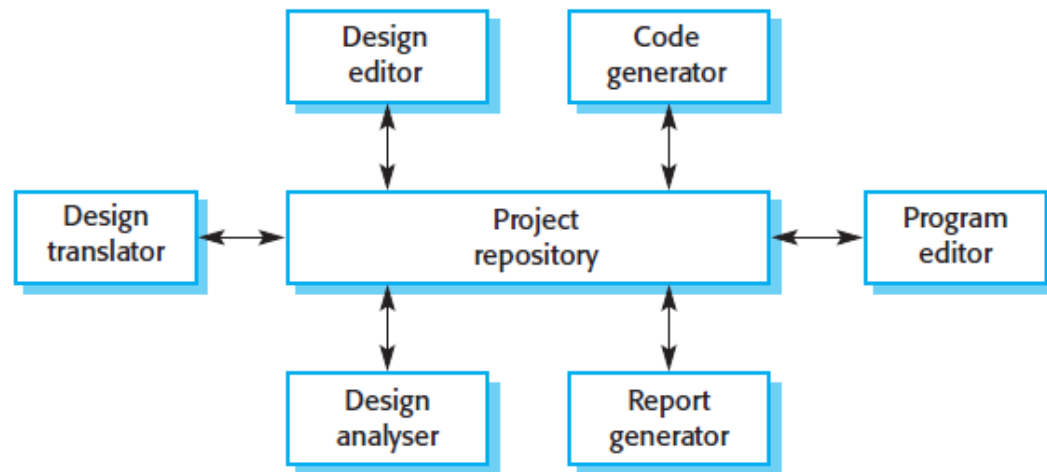
- Reflects the basic strategy that is used to structure a system.
- Three organisational styles are widely used:
 - A shared data repository style;
 - A shared services and servers style;
 - An abstract machine or layered style.

The repository model

- Sub-systems must exchange data. This may be done in two ways:
 - Shared data is held in a central database or repository and may be accessed by all sub-systems;
 - Each sub-system maintains its own database and passes data explicitly to other sub-systems.
- When large amounts of data are to be shared, the repository model of sharing is most commonly used.

Architecture of an integrated CASE toolset

Figure 11.2 The architecture of an integrated CASE toolset



Repository model characteristics

- Advantages
 - Efficient way to share large amounts of data;
 - Sub-systems need not be concerned with how data is produced Centralised management e.g. backup, security, etc.
 - Sharing model is published as the repository schema.
- Disadvantages
 - Sub-systems must agree on a repository data model. Inevitably a compromise;
 - Data evolution is difficult and expensive;
 - No scope for specific management policies;
 - Difficult to distribute efficiently.

The client–server model

- The client–server architectural model is a system model where **the system is organized as a set of services and associated servers and clients that access and use the services.**

The major components of this model are:

- **1. A set of servers that offer services to other sub-systems.** Examples of servers are print servers that offer **printing services**, file servers that offer **file management services** and a compile server, which offers **programming language compilation services.**

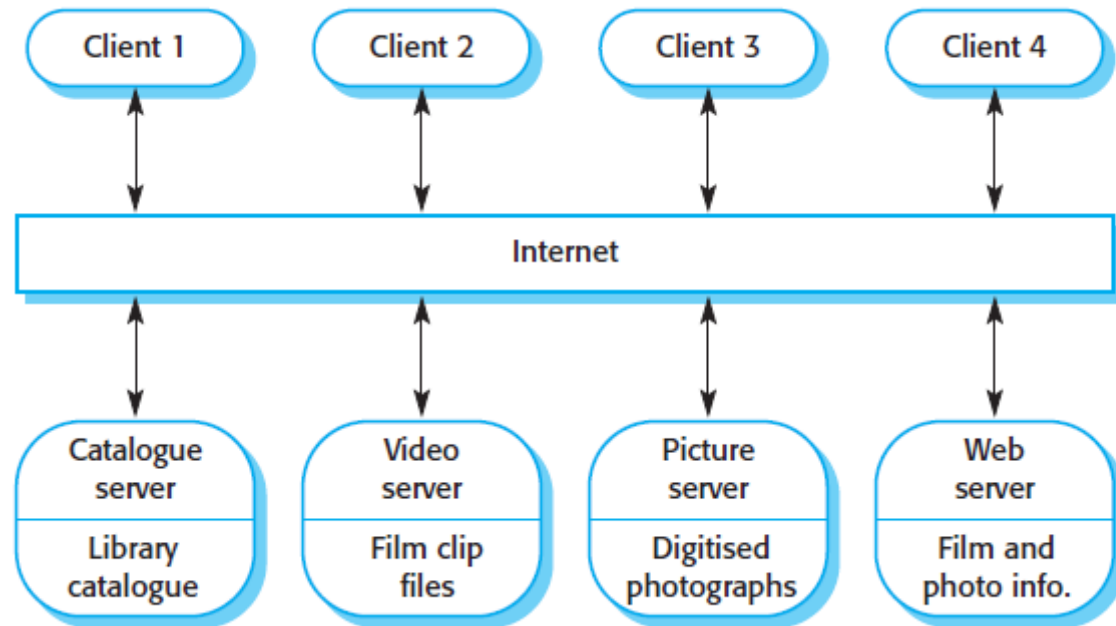
- **2. A set of clients that call on the services offered by servers.** These are normally sub-systems in their own right. There may be several instances of a client program executing concurrently.
- **3. A network that allows the clients to access these services.** This is not strictly necessary as both the clients and the servers could run on a single machine. In practice, however, most client–server systems are implemented as distributed systems.

Client-server model

- Distributed system model which shows how data and processing is distributed across a range of components.
- Set of stand-alone servers which provide specific services such as printing, data management, etc.
- Set of clients which call on these services.
- Network which allows clients to access servers.

Film and Picture Library System

Figure 11.3 The architecture of a film and picture library system



Client-server characteristics

- Advantages

- Distribution of data is straightforward;
- Makes effective use of networked systems. May require cheaper hardware;
- Easy to add new servers or upgrade existing servers.

- Disadvantages

- No shared data model so sub-systems use different data organisation. Data interchange may be inefficient;
- Redundant management in each server;
- No central register of names and services - it may be hard to find out what servers and services are available.

Abstract machine (layered) model

- Used to model the interfacing of sub-systems.
- Organises the system into a set of layers (or abstract machines) each of which provide a set of services.
- Supports the incremental development of sub-systems in different layers. When a layer interface changes, only the adjacent layer is affected.
- However, often artificial to structure systems in this way.

The layered model

- The layered model of an architecture (sometimes called an abstract machine model) organizes a system into layers, each of which provide a set of services.
- The configuration management system manages versions of objects and provides **general configuration management facilities**.

- Object management system that provides information storage and management services for configuration items or objects.
- Database system to provide basic data storage and services such as transaction management, rollback and recovery, and access control.
- The database management uses the underlying operating system facilities and file store in its implementation.

Version management system

Configuration management system layer

Object management system layer

Database system layer

Operating system layer

Modular decomposition styles

- Styles of decomposing sub-systems into modules.
- No rigid distinction between system organisation and modular decomposition.

Sub-systems and modules

- A sub-system is a system in its own right whose operation is independent of the services provided by other sub-systems.
- A module is a system component that provides services to other components but would not normally be considered as a separate system.

Modular decomposition

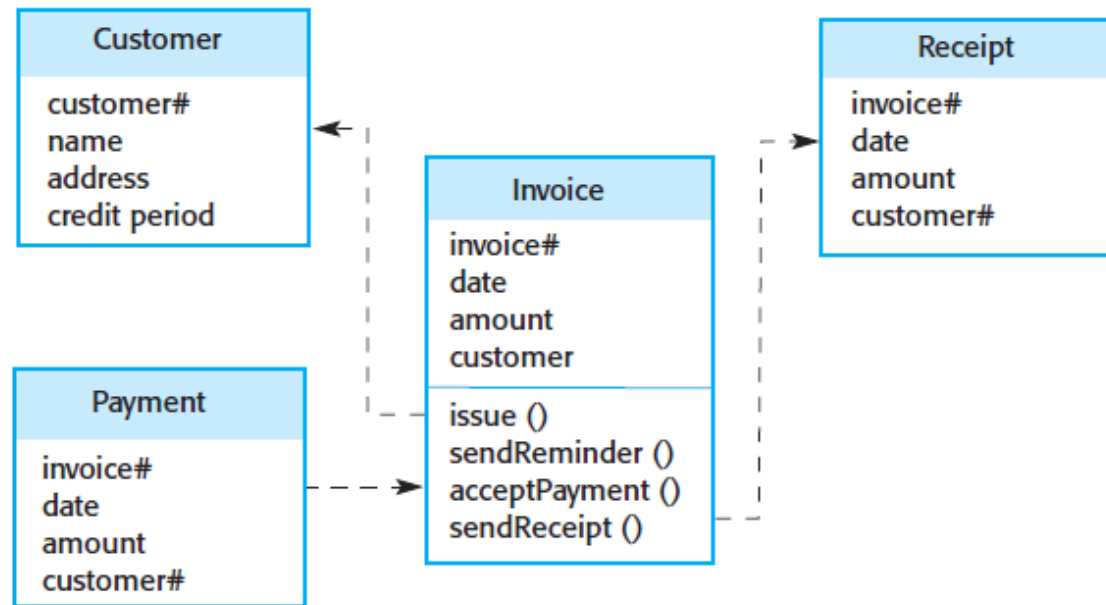
- Another structural level where sub-systems are decomposed into modules.
- Two modular decomposition models covered
 - An object model where the system is decomposed into interacting objects;
 - A pipeline or data-flow model where the system is decomposed into functional modules which transform inputs to outputs.
- If possible, decisions about concurrency should be delayed until modules are implemented.

Object models

- Structure the system into a set of loosely coupled objects with well-defined interfaces.
- Object-oriented decomposition is concerned with identifying object classes, their attributes and operations.
- When implemented, objects are created from these classes and some control model used to coordinate object operations.

Invoice Processing System

Figure 11.5 An object model of an invoice processing system



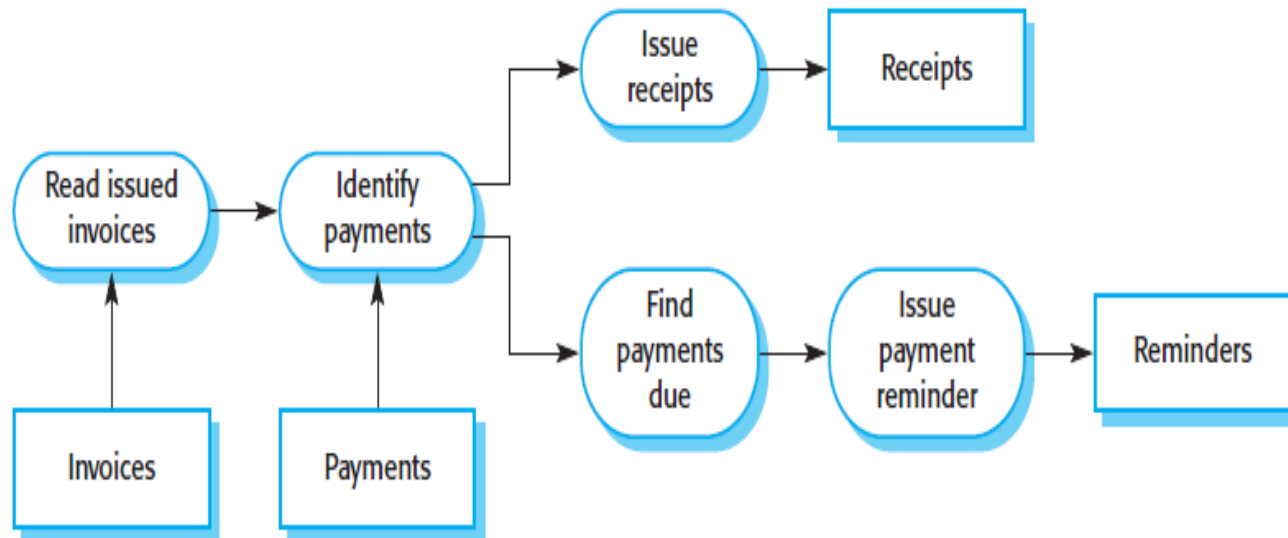
Object model advantages

- Objects are loosely coupled so their implementation can be modified without affecting other objects.
- The objects may reflect real-world entities.
- OO implementation languages are widely used.
- However, object interface changes may cause problems and complex entities may be hard to represent as objects.

Function-oriented pipelining

- Functional transformations process their inputs to produce outputs.
- May be referred to as a pipe and filter model (as in UNIX shell).
- Variants of this approach are very common. When transformations are sequential, this is a batch sequential model which is extensively used in data processing systems.
- Not really suitable for interactive systems.

A pipeline model of an Invoice Processing System



Pipeline model advantages

- Supports transformation reuse.
- Intuitive organisation for stakeholder communication.
- Easy to add new transformations.
- Relatively simple to implement as either a concurrent or sequential system.
- However, requires a common format for data transfer along the pipeline and difficult to support event-based interaction.

Control styles

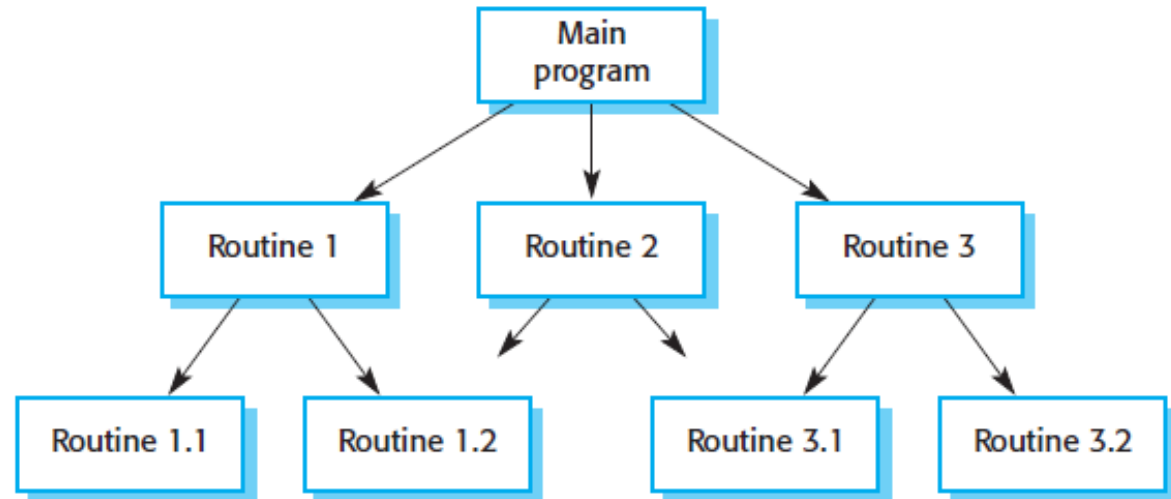
- Are concerned with the control flow between sub-systems. Distinct from the system decomposition model.
- Centralised control
 - One sub-system has overall responsibility for control and starts and stops other sub-systems.
- Event-based control
 - Each sub-system can respond to externally generated events from other sub-systems or the system's environment.

Centralised control

- A control sub-system takes responsibility for managing the execution of other sub-systems.
- Call-return model
 - Top-down subroutine model where control starts at the top of a subroutine hierarchy and moves downwards. Applicable to sequential systems.
- Manager model
 - Applicable to concurrent systems. One system component controls the stopping, starting and coordination of other system processes. Can be implemented in sequential systems as a case statement.

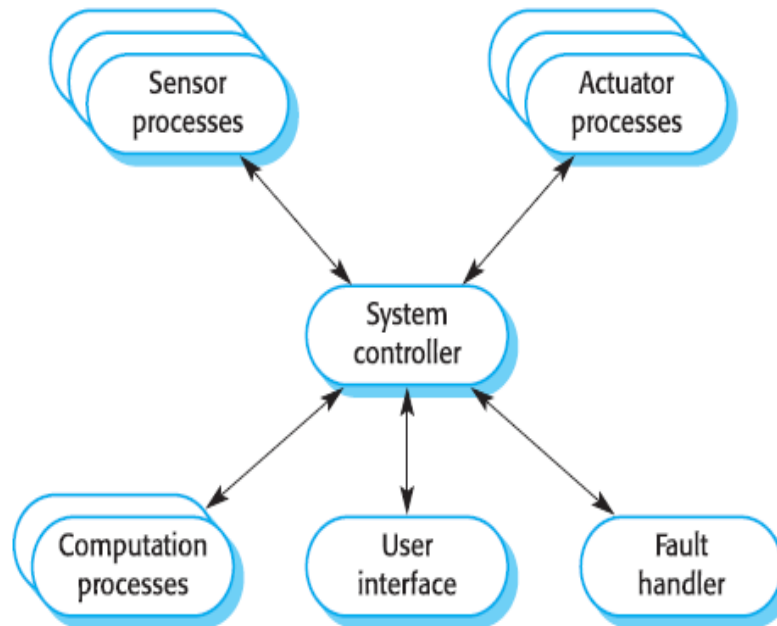
Call return Model

Figure 11.7 The call-return model of control



Real Time System Control

Figure 11.8 A
centralised control
model for a real-time
system



Event-driven systems

- Driven by externally generated events where the timing of the event is outwith the control of the sub-systems which process the event.
- Two principal event-driven models
 - Broadcast models. An event is broadcast to all sub-systems. Any sub-system which can handle the event may do so;
 - Interrupt-driven models. Used in real-time systems where interrupts are detected by an interrupt handler and passed to some other component for processing.
- Other event driven models include spreadsheets and production systems.

Event-driven systems

- Event-driven control models are driven by **externally generated events**.
- The term *event* in this context does not just mean a binary signal.
- There are many types of event-driven systems. These include editors where user interface events signify editing commands.

1. *Broadcast models* In these models, an **event is broadcast to all sub-systems**. Any sub-system that has been programmed to handle that event can respond to it.

2. *Interrupt-driven models* These are exclusively used in real-time systems where external interrupts are detected by an interrupt handler. They are then passed to some other component for processing.

Broadcast Model

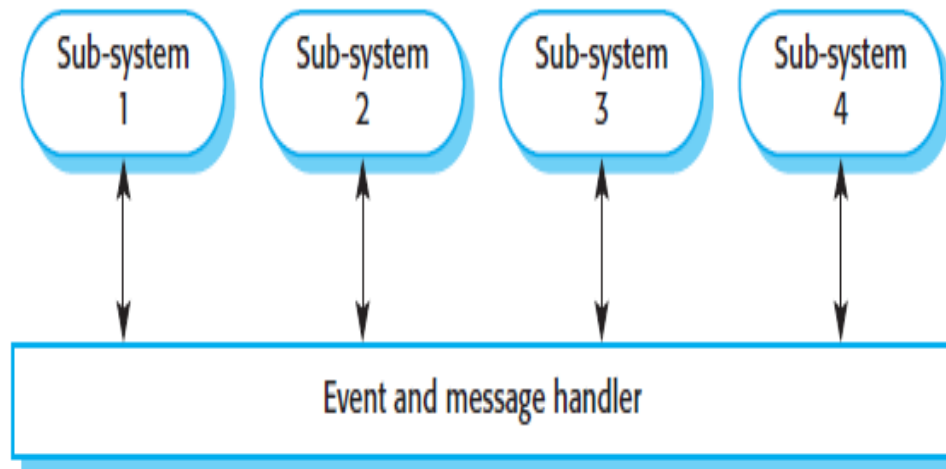
- **All events could be broadcast to all sub-systems**, but this imposes a great deal of processing overhead.
- More often, **the event and message handler maintains a register**

of sub-systems and the events of interest to them. Sub-systems generate events

indicating, perhaps, that some data is available for processing. The event handler detects the events, consults the event register and passes the event to those subsystems

Selective Broadcasting

Figure 11.9 A control model based on selective broadcasting



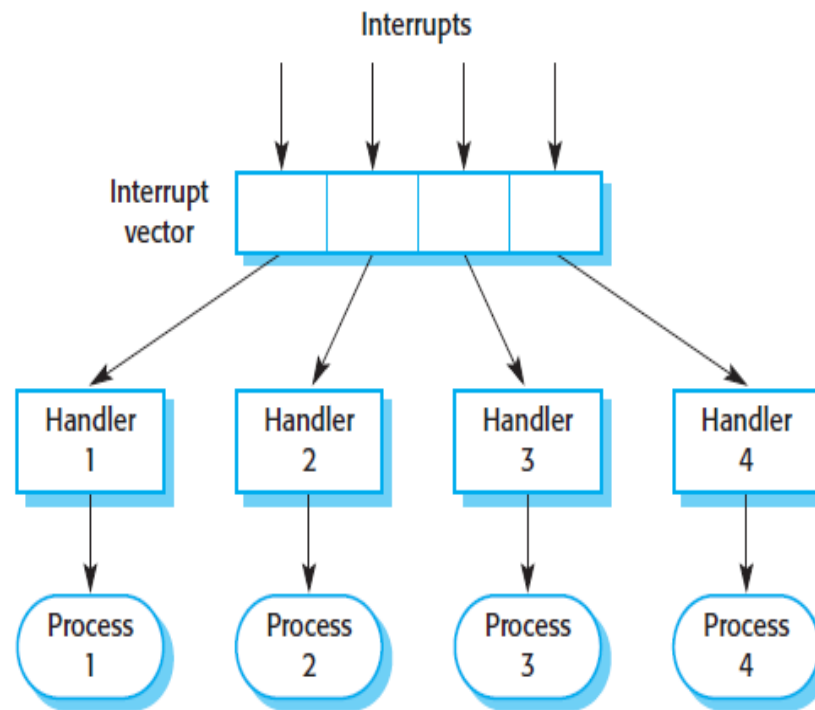
Interrupt driven Control Model

- There are a known number of **interrupt types with a handler** defined for each type.
- Each type of **interrupt is associated with the memory location where its handler's address is stored.**

- When an interrupt of a particular type is received, a hardware switch causes control to be transferred immediately to its handler. This interrupt handler may then start or stop other processes in response to the event signaled by the interrupt.
- This model is mostly used **in real-time systems** where an immediate response to some event is necessary.

Interrupt driven Control Model

Figure 11.10 An interrupt-driven control model



Interrupt driven Control Model

- The advantage of this approach is that it allows very fast responses to events to be implemented.
- Its disadvantages are that it is complex to program and difficult to validate.

Key points

- The software architecture is the fundamental framework for structuring the system.
- Architectural design decisions include decisions on the application architecture, the distribution and the architectural styles to be used.
- Different architectural models such as a structural model, a control model and a decomposition model may be developed.
- System organisational models include repository models, client-server models and abstract machine models.

Key points

- Modular decomposition models include object models and pipelining models.
- Control models include centralised control and event-driven models.
- Reference architectures may be used to communicate domain-specific architectures and to assess and compare architectural designs.